Python Programming
Unit 01 – Lecture 03 Notes
Tokens, Naming, Strings, and Numeric Types

Tofik Ali

February 17, 2026

Contents

1 Lecture Overview

This lecture focuses on the **building blocks** of Python code:

- tokens (keywords, identifiers, literals, operators),
- naming conventions for readable programs,
- strings (methods, operators, formatting),
- and basic numeric data types (`int`, `float`, `complex`).

2 Core Concepts

2.1 Python Tokens

A **token** is a small unit of a program. Common token categories:

- **Keywords:** reserved words such as `if`, `for`, `while`, `def`.
- **Identifiers:** names you create (variables, functions, classes).
- **Literals:** fixed values like `10`, `3.14`, `"hello"`.
- **Operators:** symbols/words that perform operations: `+`, `-`, `*`, `and`, `==`.
- **Delimiters:** punctuation used in syntax: `( ) [ ] { } , :`.

2.2 Identifiers and Naming Conventions

Rules (practical):

- Start with a letter or underscore; remaining characters can include digits.
- Identifiers are case-sensitive: `Name` and `name` are different.
- Do not use keywords as identifiers.

Conventions (recommended):

- Use **snake_case** for variables and functions: `total_marks`, `calculate_cgpa`.
- Choose meaningful names. Prefer `marks` over `m`.

2.3 Strings: Values, Indexing, Slicing

A **string** is a sequence of characters, written with quotes.

```
s1 = "Python"
s2 = 'UPES'
```

Strings support indexing and slicing:

```
s = "python"
s[0] # 'p'
s[-1] # 'n'
s[1:4] # 'yth'
```

Note: strings are **immutable**. Methods usually return a new string.

2.4 Common String Methods

Useful beginner methods:

- `lower()`, `upper()`, `title()`
- `strip()` removes leading/trailing whitespace
- `split()` splits into a list (by spaces by default)

- `replace(old, new)` replaces occurrences
- `find(sub)`, `count(sub)`

```
text = " UPES Dehradun "
clean = text.strip()
print(clean) # "UPES Dehradun"
print(clean.lower()) # "upes dehradun"
print(clean.split()) # ["UPES", "Dehradun"]
```

2.5 String Operators

Strings support some operators:

- **Concatenation** using `+`
- **Repetition** using `*`
- **Membership** using `in`

```
print("py" * 3) # pypypy
print("th" in "python") # True
```

2.6 Formatting Output: `format()` and f-strings

Formatting is important for readable output (grade sheets, reports, logs).

`format()` method

```
name = "Rohit"
cgpa = 7.0
print("Name: {}, CGPA: {:.1f}".format(name, cgpa))
```

`:.1f` means "one digit after the decimal point".

f-strings

f-strings are short and readable:

```
print(f"Name: {name}, CGPA: {cgpa:.1f}")
```

2.7 Numeric Data Types

Main numeric types in this unit:

- `int`: integers (no decimals)
- `float`: real numbers with decimals
- `complex`: numbers like `2+3j`

Type conversions (very common):

```
x = int("10") # 10
y = float("2.5") # 2.5
```

3 Demo Walkthrough

File: `demo/string_formatting_demo.py`

Goal

Read user text, clean extra spaces, compute a small summary, and print a nicely formatted line.

Run

```
python demo/string_formatting_demo.py
```

4 Interactive Checkpoints (with Solutions)

Checkpoint 1 Solution

Question: output of `print("py" * 3)`?
Answer: `pypypy`

Checkpoint 2 Solution

Question: difference between `split()` and `join()`?
Answer:

- `split()` breaks a string into a list.
- `join()` joins a list of strings into a single string.

```
sentence = "Python is fun"
parts = sentence.split() # ["Python", "is", "fun"]
again = "-".join(parts) # "Python-is-fun"
```

5 Practice Exercises (with Solutions)

Exercise 1: Clean a String

Task: Read a string and print it without leading/trailing spaces.
Solution:

```
s = input("Enter text: ")
print(s.strip())
```

Exercise 2: Count Vowels

Task: Count vowels in an input string (a, e, i, o, u; case-insensitive).
Solution:

```
s = input("Enter text: ").lower()
vowels = "aeiou"
count = 0
for ch in s:
    if ch in vowels:
        count += 1
print("Vowel count =", count)
```

Exercise 3: Format to Two Decimals

Task: Print a floating value with 2 decimal places.

Solution (f-string):

```
x = 3.14159
print(f"{x:.2f}")
```

6 Exit Question (with Solution)

Question: Format `3.14159` to 2 decimal places.

Answer: `3.14`. One way:

```
print(f"{3.14159:.2f}")
```

Appendix: Slide Deck Content (Reference)

The material below is a reference copy of the slide deck content. Exercise solutions are explained in the main notes where applicable.

Title Slide

Repository: `https://github.com/tali7c/Python-Programming`

Quick Links

Core Concepts Demo Interactive Summary

Agenda

- Core Concepts
- Demo
- Interactive
- Summary

Learning Outcomes

- Identify Python tokens (keywords, identifiers, literals, operators)
- Follow good naming conventions (snake_case)
- Use common string methods and operators
- Format output using `format()` / f-strings
- Work with numeric types (`int`, `float`, `complex`)

Python Tokens (Big Picture)

- **Keywords**: reserved words (e.g., `if`, `for`, `def`)
- **Identifiers**: names you create (variables, functions)
- **Literals**: fixed values (`10`, `3.14`, `"hi"`)
- **Operators**: `+ - * / == <` etc.
- **Delimiters**: `( ) [ ] { } , :`

Identifiers and Naming Conventions

- Use **snake_case**: `total_marks`, `student_name`
- Names should be meaningful and consistent
- Do not use keywords as names (e.g., `for = 5` is invalid)

String Basics

- Strings are sequences of characters (immutable)
- Indexing and slicing works like lists

```
s = "python"
print(s[0]) # p
print(s[-1]) # n
print(s[1:4]) # yth
```

Common String Methods

- `lower()`, `upper()`, `title()`
- `strip()` (remove leading/trailing spaces)
- `split()` (string → list of words)
- `replace(old, new)`
- `find(sub)` / `count(sub)`

String Operators

- Concatenation: `"Py" + "thon"` → `"Python"`
- Repetition: `"py" * 3` → `"pypypy"`
- Membership: `"th" in "python"` → `True`

Formatting Output

- `format()` method:

```
name = "Rohit"
cgpa = 7.0
print("Name: {}, CGPA: {:.1f}".format(name, cgpa))
```

- f-strings (recommended for readability):

```
print(f"Name: {name}, CGPA: {cgpa:.1f}")
```

Numeric Data Types

- `int`: integers (e.g., `5`, `-12`)
- `float`: decimals (e.g., `3.14`)
- `complex`: complex numbers (e.g., `2+3j`)
- Convert types when needed: `int("10")`, `float("2.5")`

Demo: Cleaning and Formatting Strings

- File: `demo/string_formatting_demo.py`
- Reads a name and marks, cleans extra spaces
- Prints a formatted student summary line

Checkpoint 1

Question: What is the output?

```
print("py" * 3)
```

Checkpoint 2

Question: What is the difference between `split()` and `join()`?

```
sentence = "Python is fun"
parts = sentence.split()
again = "-".join(parts)
```

Think-Pair-Share

You have input text with extra spaces:

- `" UPES Dehradun "`

Discuss: which methods would you use to clean it?

Key Takeaways

- Tokens build programs: keywords, identifiers, literals, operators
- Use clean naming: snake_case, meaningful names
- Strings are immutable; methods return new strings
- Use `split/join/strip` for text processing
- f-strings and `format()` produce clean output

Exit Question

Format `3.14159` to **2 decimal places**.